

BUG #1: SQL Boolean Operator Precedence Bug

1. WHERE THE BUG IS

- File: backend/src/main/java/com/internal/tasktracker/TaskRepository.java (Lines 14-16)
- Also in: db/queries/search_tasks.sql (Lines 10-13) & db/oracle/task_search_package.sql
- Layer: Database / Persistence / SQL Query Layer

2. HOW I DISCOVERED IT

- Step 1: Searched term 'api' in UI. Expected 6 active tasks, but saw 8 tasks returned.
- Step 2: Found archived tasks #20 ('Decommission legacy...') & #21 ('Legacy API...') present!
- Step 3: Filtered by Status='DONE' with search 'Fix' -> OPEN/IN_PROGRESS tasks still showed up!

3. ROOT CAUSE ANALYSIS

- In SQL Standard, AND operator has higher precedence than OR operator.
- Original unparenthesized query:

```
WHERE archived = FALSE AND LOWER(title) LIKE :term  
OR LOWER(description) LIKE :term AND (:status IS NULL OR status = :status)
```

- SQL parser evaluates this as TWO separate branches:
Branch A: (archived = FALSE AND LOWER(title) LIKE :term)
-> If title matches, the status filter is completely BYPASSED!
Branch B: (LOWER(description) LIKE :term AND (:status IS NULL OR status = :status))
-> If description matches, archived = FALSE check is BYPASSED (leaks archived data)!

4. HOW I FIXED IT & WHY

- Added explicit parentheses around the OR search criteria:

```
SELECT * FROM tasks  
WHERE archived = FALSE  
AND (LOWER(title) LIKE :term OR LOWER(description) LIKE :term)  
AND (:status IS NULL OR status = :status) ORDER BY created_at DESC
```

- Why this approach: It strictly enforces both archived=FALSE and status filter across all text matches with zero overhead and no breaking schema changes.

BUG #2: Backend Latency, Enum Crash & Pagination Bounds

1. WHERE THE BUG IS

- File: backend/src/main/java/com/internal/tasktracker/TaskController.java
- Lines: 31-33 (enum parsing), 36-42 (Thread.sleep), 50-54 (subList pagination)
- Layer: Backend REST Controller / API Request Handling Layer

2. HOW I DISCOVERED IT

- Step 1: Noticed extreme 1000ms sluggishness in UI whenever typing 1-2 character search terms.
- Step 2: Tested invalid status parameter curl /api/tasks?status=FOO -> crashed with HTTP 500.
- Step 3: Tested negative/zero page curl /api/tasks?page=0 -> threw IndexOutOfBoundsException.

3. ROOT CAUSE ANALYSIS

- Root Cause A: Thread.sleep(Math.max(0, 10 - query.length()) * 100L) blocked worker threads. Shorter queries waited 1000ms while longer queries finished in 0ms, causing race conditions.
- Root Cause B: TaskStatus.valueOf(status.toUpperCase()) throws IllegalArgumentException when given invalid status, uncaught -> Spring returns unhandled 500 Internal Server Error.
- Root Cause C: int start = (page - 1) * pageSize -> for page=0, start = -10 (invalid index).

4. HOW I FIXED IT & WHY

```
// 1. Safe Enum Parsing -> returns 400 Bad Request on invalid input
if (status != null && !status.trim().isEmpty()) {
    try { normalizedStatus = TaskStatus.valueOf(status.trim().toUpperCase()).name(); }
    catch (IllegalArgumentException e) { return ResponseEntity.badRequest()...; } }
// 2. Removed Thread.sleep completely
// 3. Bounds Guard: int validPage = Math.max(1, page); int validSize = Math.max(1, pageSize);
int start = (validPage - 1) * validSize; int end = Math.min(start + validSize, allResults.size());
```

- Rationale: Eliminated artificial latency, hardened API against malformed inputs, prevented 500 server crashes, and guaranteed non-negative subList offsets.

BUG #3: Frontend Race Conditions & Stuck Loading State

1. WHERE THE BUG IS

- File: frontend/src/hooks/useTasks.js (Lines 10-22) & frontend/src/api.js
- Layer: Frontend Asynchronous State Management / React Hooks Layer

2. HOW I DISCOVERED IT

- Step 1: Typed fast into search bar (e.g. 'rate'). Results flickered erratically.
- Step 2: Observed network tab: Request 1 ('r') returned AFTER Request 4 ('rate').
- Step 3: State 'r' results overwrote 'rate' results in state! Final UI showed wrong data.
- Step 4: When API threw 400/500 error, UI was stuck in 'Loading tasks...' permanently.

3. ROOT CAUSE ANALYSIS

- Out-of-order execution: In async JS, promise resolution order != dispatch order.
- useTasks had no AbortController or ignore flag to cancel/ignore stale requests.
- Error handling flaw: .catch((err) => setError(err.message)) omitted setLoading(false).

4. HOW I FIXED IT & WHY

```
useEffect(() => {  
  let ignore = false; const controller = new AbortController();  
  setLoading(true); setError(null);  
  fetchTasks({ query, status, page, pageSize, signal: controller.signal })  
    .then(data => { if (!ignore) { setTasks(data.items); setTotal(data.total); setLoading(false); }})  
    .catch(err => {  
      if (err.name === 'AbortError') return;  
      if (!ignore) { setError(err.message); setLoading(false); }});  
  return () => { ignore = true; controller.abort(); };  
});
```

- Rationale: Cancels in-flight stale network requests, prevents unmounted state updates, resets errors cleanly on retry, and ensures loading spinner always terminates.

BUG #4: Frontend Pagination Desynchronization on Filter

1. WHERE THE BUG IS

- File: frontend/src/App.jsx (Lines 8-26)
- Layer: Frontend Application State / UI Component Layer

2. HOW I DISCOVERED IT

- Step 1: Navigated to Page 3 of all tasks (tasks #21-30).
- Step 2: Typed 'rate limiting' into the search input (matching only 1 total task).
- Step 3: UI showed 'No tasks found.' with pagination label 'Page 3 of 1'!
- Step 4: Same issue occurred when changing StatusFilter dropdown on high page numbers.

3. ROOT CAUSE ANALYSIS

- The 'page' state was independent of 'query' and 'status' state.
- When query or status changed, page remained at its old value (e.g. 3).
- The backend calculated start = $(3 - 1) * 10 = 20$. Since total results = 1, subList returned empty items [], leading to a phantom empty state.

4. HOW I FIXED IT & WHY

```
const handleQueryChange = (newQuery) => {  
  setQuery(newQuery);  
  setPage(1); // Reset to page 1 on new search  
};  
const handleStatusChange = (newStatus) => { setStatus(newStatus); setPage(1); };
```

- Created dedicated handleQueryChange & handleStatusChange event handlers in App.jsx.
- Rationale: Immediate, explicit reset on user interaction; zero side effects, no extra useEffects.

BUG #5: Oracle PL/SQL Package Precedence & Offset Validation

1. WHERE THE BUG IS

- File: db/oracle/task_search_package.sql (Lines 45-75)
- Layer: Oracle Database / Stored Procedure (PL/SQL Reference Artifact)

2. HOW I DISCOVERED IT

- Step 1: Conducted full architectural audit of SQL artifacts across repository.
- Step 2: Examined task_search_pkg body in task_search_package.sql.
- Step 3: Spotted identical operator precedence flaw in both COUNT(*) and OPEN p_results cursor.
- Step 4: Spotted unprotected v_offset calculation when p_page < 1 or NULL.

3. ROOT CAUSE ANALYSIS

- In Oracle SQL, unparenthesized OR conditions cause archived tasks (archived = 1) to leak whenever description matches v_term, and bypasses p_status when title matches.
- $v_offset := (p_page - 1) * p_page_size$ yields negative numbers for $page \leq 0$.

4. HOW I FIXED IT & WHY

```
-- 1. Safe Offset with GREATEST / NVL  
v_offset := (GREATEST(NVL(p_page, 1), 1) - 1) * GREATEST(NVL(p_page_size, 10), 1);  
-- 2. Enforce Operator Precedence in COUNT and Cursor  
WHERE archived = 0  
AND (LOWER(title) LIKE v_term OR LOWER(description) LIKE v_term)  
AND (p_status IS NULL OR status = p_status)
```

- Rationale: Ensures production parity between H2 Spring Data query and Oracle PL/SQL.